

When Does INT8 Actually Accelerate Host-CPU Inference?

A Reproducible Audit of MobileNetV2 Conversion and Packaging

Jaiveer Bassi 

Independent Researcher, USA

jaiveerbassi@yahoo.com

ORCID: 0009-0006-3633-3220

Abstract—Eight-bit quantization is often presented as a direct route to smaller and faster edge models, but bit width alone does not determine realized latency. We audit a reproducible MobileNetV2 TensorFlow Lite host-CPU pipeline across quantization granularity, CPU backend, thread count, batch size, process restarts, and container serving. On the complete 3,925-image Imagenette validation split, per-channel INT8 reduced storage by 71.5% and changed top-1 accuracy by -0.92 percentage points (paired bootstrap 95% CI $[-1.66, -0.18]$; exact McNemar $p = .0165$). Across 20 fresh processes on an AMD Ryzen 5 7600, one-thread XNNPACK produced a $1.27\times$ paired geometric-mean speedup (95% CI $[1.20, 1.35]$), whereas TensorFlow Lite’s built-in-without-default-delegates path made INT8 $2.24\times$ slower (speedup $0.45\times$, $[0.42, 0.47]$). Per-tensor INT8 had similar latency but collapsed to 1.04% top-1 accuracy and concentrated 19.6% of predictions in one class. The slim container occupied 339.58 MiB uncompressed (81.43 MiB compressed), reached health in a 1.52 s median across 20 starts, and delivered localhost HTTP p50/p99 latency of 49.87/73.96 ms. The central finding is that backend choice and quantization granularity govern whether INT8 is both accurate and fast; this host-side audit does not claim ARM-device performance.

Index Terms—edge inference, integer quantization, MobileNetV2, TensorFlow Lite, benchmarking, containerization

I. INTRODUCTION

Executing inference near the data source can reduce network dependence and response time, but edge devices impose tight limits on storage, memory, computation, and power. Integer post-training quantization (PTQ) is attractive because it can reduce model storage by approximately fourfold and enable integer kernels [1], [2]. This storage argument follows directly from representation width; the latency argument does not. Realized speed depends on graph structure, kernel coverage, delegate selection, threading, and hardware [3]–[5].

MobileNetV2 is a useful audit subject: it is a canonical compact architecture built from depthwise separable convolutions and inverted residual blocks [6], and its PTQ workflow is mature enough that conversion alone is not a research contribution. The interesting question is therefore narrower and testable: under what runtime and quantization conditions does an otherwise fixed INT8 model remain accurate and become faster?

This work contributes an empirical audit and its public executable artifact, not a new quantization algorithm. It is

intentionally host-side: x86 CPU and container measurements isolate software-stack effects before future replication on ARM hardware. It makes three claims. First, per-channel INT8 preserves useful accuracy while a per-tensor control does not. Second, INT8 speedup is conditional: it is modest with the default XNNPACK delegate and reverses on a built-in-kernel counterfactual. Third, packaging costs are measurable system properties rather than prose-level design details. The artifact supplies paired predictions, raw timings from independent processes, model and dataset hashes, container measurements, and one entry point that regenerates every reported quantity.

II. RELATED WORK AND RESEARCH QUESTIONS

Integer-only inference has a well-established algorithmic basis. Jacob et al. described affine quantization, integer arithmetic, and training procedures that made eight-bit deployment practical across mobile models [1]. Subsequent tooling made PTQ a routine conversion path, but the convenience of conversion can obscure two distinct validation problems: whether the quantized graph preserves predictions and whether the selected runtime has efficient kernels for that graph. MQBench formalized the need to separate simulated quantization quality from deployable backend behavior [4]. SpaceEvo similarly showed that INT8 latency is architecture- and hardware-dependent, motivating search spaces designed around actual low-precision execution rather than floating-point proxies [5].

Benchmark suites address a complementary systems problem. MLPerf Inference, Mobile, and Tiny specify scenarios and measurement rules because latency numbers are otherwise inseparable from hardware, software, and load assumptions [3], [7], [8]. This study is smaller in model and platform scope, but applies the same principle to an inspectable end-to-end artifact. It also distinguishes native interpreter latency from HTTP and container measurements, which have different timed boundaries.

The audit is organized around three research questions. *RQ1*: Does full-integer PTQ preserve accuracy under per-channel and per-tensor weight quantization? *RQ2*: How does the FP32/INT8 latency ratio change with delegate, thread count, and batch size when uncertainty is estimated across fresh processes? *RQ3*: What image-size, cold-start, memory, and request-latency costs accompany the deployable container?

These questions turn a familiar conversion recipe into falsifiable claims about correctness and realized system behavior.

III. METHODOLOGY

A. Models and quantization controls

The source model is MobileNetV2 with ImageNet weights and a $224 \times 224 \times 3$ input. TensorFlow 2.15.1 exports the same graph as FP32 and as signed full-integer TFLite. The primary INT8 conversion uses per-channel weight scales; a control disables per-channel quantization. Both use 200 deterministically selected training images to calibrate activation ranges, restrict operators to the built-in INT8 set, and request signed INT8 inputs and outputs. For scale S , zero point Z , and integer bounds $q_{\min}$ and $q_{\max}$, an input value x is mapped as

$$q = \text{clip}(\text{round}(x/S) + Z, q_{\min}, q_{\max}). \quad (1)$$

All three outputs are retained with SHA-256 hashes. Tensor metadata verify that the control has at most one scale per tensor, while the primary model contains 104 tensors with multiple scales. Because per-channel scales are particularly important for depthwise MobileNet layers [9], diagnostics also retain the per-tensor model’s prediction histogram and channel-range statistics for every depthwise weight tensor.

B. Dataset and paired accuracy protocol

Imagenette is a ten-class subset of ImageNet containing tench, English springer, cassette player, chain saw, church, French horn, garbage truck, gas pump, golf ball, and parachute [10]. The 160-pixel distribution contains 9,470 training and 3,925 validation images. Its archive SHA-256 is 64d0c4859f35a461889e0147755a999a48b49bf38a7e0f9bd27003f10db02fe5. Calibration images are selected from the training split with seed 20260914. All validation images are evaluated.

Each image is converted to RGB, resized so its shortest side is 256 pixels, center-cropped to 224×224 , and scaled to $[-1, 1]$. Predicted indices are compared with the corresponding ImageNet class indices. Because FP32 and INT8 see the same images, the primary comparison is paired. We report the discordant counts, the exact two-sided McNemar test [11], and a percentile interval for the accuracy difference from 100,000 paired bootstrap resamples with seed 20260914 [12]. Marginal Wilson intervals remain in the machine-readable results but are not used to test a difference. This is a conversion-fidelity study on an ImageNet subset, not an Imagenette training benchmark.

C. Repeated-process latency protocol

Latency is measured on an AMD Ryzen 5 7600 (6 physical/12 logical cores) running Microsoft Windows 24H2, build 26100.9278, and TensorFlow Lite 2.15.1 with oneDNN disabled. XNNPACK is the Python interpreter’s default CPU delegate. The non-delegate condition selects TensorFlow Lite 2.15.1’s `BUILTIN_WITHOUT_DEFAULT_DELEGATES` resolver; it is a counterfactual kernel path, not a recommended deployment configuration. Every replicate starts a fresh process and interpreter, uses 50 untimed warm-ups, then times 500

deterministically shuffled images with a monotonic nanosecond clock; the slower counterfactual uses 100 images. Preprocessing is excluded. Twenty matched replicates alternate FP32/INT8 order within deterministically shuffled configuration blocks. Benchmark processes use above-normal priority and physical-core-spaced affinity. Docker Desktop is stopped and the Windows Balanced power plan is active. Interactive applications deliberately remain open to represent a developer-workstation deployment host rather than an idle laboratory machine; pre-interpreter host CPU load is retained for every run.

The controls cross XNNPACK thread counts $\{1, 2, 4, 6\}$, XNNPACK versus the built-in counterfactual at one thread, and batch one versus batch four. A per-tensor INT8 latency control is also run. For each model we report the mean of 20 process means with a two-sided Student- t 95% interval. For matched FP32/INT8 pairs we report the geometric mean of replicate speedup ratios and a t interval in log space (19 degrees of freedom).

D. Serving and packaging protocol

The FastAPI service loads the per-channel INT8 model with `tf.lite-runtime` 2.14.0, validates uploads, applies the same preprocessing, reads quantization parameters from tensor metadata, serializes interpreter access, and exposes health and metadata endpoints. Docker Desktop 4.91.0 (Engine 29.8.0, Compose 5.5.1) runs the Linux/amd64 image on the Windows host as UID 10001 with a read-only root filesystem, 512 MiB memory ceiling, and two-CPU ceiling.

Packaging is measured separately from native inference. Image size is Docker’s uncompressed layer size; compressed size is a `docker image save` archive compressed with deterministic gzip level 9. Identical application layers are built from pinned slim and standard Python 3.10 Bookworm bases. Twenty cold-start samples measure host wall time from compose up to the first successful health response. After 20 warm-up requests, 200 sequential `localhost /predict` requests measure end-to-end HTTP latency. Monotonic server timers separate framework parsing, upload read, decode, resize/crop, normalization/quantization, queue wait, invocation, postprocessing, serialization, and unattributed framework time. Client wall time minus server total is reported as loopback/transport overhead. Resident set size is sampled after those requests. The request input and all raw observations are retained.

IV. RESULTS

A. Accuracy depends on granularity

Table I shows that per-channel INT8 retained most FP32 accuracy while reducing storage from 13.34 to 3.81 MiB (71.5%). The top-1 comparison is not merely two overlapping marginal intervals: among the same 3,925 images, 125 were correct only for FP32 and 89 only for INT8. The resulting -0.92 percentage-point difference has paired bootstrap 95% CI $[-1.66, -0.18]$ and exact McNemar $p = .0165$. Top-5 changed by -0.94 points (76 FP32-only versus 39 INT8-only; CI $[-1.48, -0.41]$; $p = .00072$).

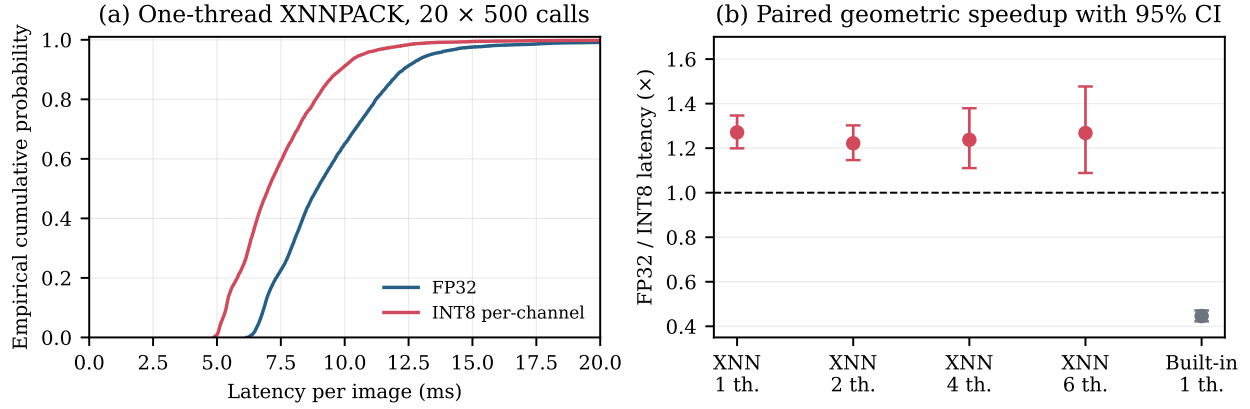


Fig. 1. Latency audit. (a) Pooled one-thread XNNPACK empirical CDFs contain 10,000 observations per precision from 20 process restarts. (b) Points are paired geometric-mean FP32/INT8 speedups; bars are log- t 95% intervals across 20 matched process means. Values above one favor INT8.

TABLE I
MODEL SIZE AND IMAGENETTE ACCURACY ($n = 3,925$).

Model	MiB	Top-1	Top-5
FP32	13.34	66.73%	90.75%
INT8, per-channel	3.81	65.81%	89.81%
INT8, per-tensor	3.42	1.04%	3.95%

The per-tensor control was 10.2% smaller than per-channel INT8 but achieved only 1.04% top-1 and 3.95% top-5 accuracy. Its one-thread XNNPACK latency (7.35 ms; 95% CI [7.00, 7.69]) was close to per-channel INT8. Yet it predicted only 187 of 1,000 classes across 3,925 images, placed 19.6% of predictions in one class, and had prediction entropy 5.11 bits versus a 9.97-bit maximum. In the valid per-channel graph, 17 depthwise weight tensors had within-layer scale ratios as high as 742 $\times$ (channel-range ratio 754 $\times$). The same evaluator and preprocessing produce normal per-channel accuracy, supporting numerical granularity—not an evaluation bug—as the cause of collapse. Quantization granularity is therefore part of model correctness, not a packaging detail.

B. Speedup depends on the execution backend

The one-thread XNNPACK condition is the primary latency comparison. FP32 and per-channel INT8 mean-of-run-means were 9.48 ms (95% CI [8.93, 10.02]) and 7.43 ms ([7.14, 7.71]), respectively. Matched run ratios yield a 1.27 $\times$ geometric-mean speedup ([1.20, 1.35]). The pooled empirical distributions in Fig. 1 show substantial jitter; inference-level samples characterize tails, while the interval is deliberately computed across process-level means.

Table II demonstrates that this result does not extrapolate to a universal multiplier. XNNPACK speedups range from 1.22 $\times$ to 1.27 $\times$ across thread counts; all paired intervals exclude unity after 20 restarts. Batch four increases the paired speedup to 1.38 $\times$ but does not materially reduce FP32 per-image latency. Most importantly, disabling default delegates reverses the result: built-in FP32 averages 34.59 ms and built-

TABLE II
MATCHED LATENCY CONTROLS. MEANS ARE MILLISECONDS PER IMAGE ACROSS 20 FRESH PROCESSES; SPEEDUP IS FP32/INT8 WITH PAIRED 95% CI.

Condition	FP32	INT8	Speedup [95% CI]
XNNPACK, 1 thread	9.48	7.43	1.27 [1.20, 1.35]
XNNPACK, 2 threads	4.30	3.54	1.22 [1.15, 1.30]
XNNPACK, 4 threads	2.51	2.04	1.24 [1.11, 1.38]
XNNPACK, 6 threads	2.16	1.74	1.27 [1.09, 1.48]
Built-in, 1 thread	34.59	77.27	0.45 [0.42, 0.47]
XNNPACK, batch 4	9.86	7.16	1.38 [1.31, 1.46]

in INT8 averages 77.27 ms, making INT8 2.24 $\times$ slower. Thus, the optimized kernel path—not the file’s nominal bit width—determines whether the conversion accelerates inference. This observation is consistent with benchmark literature that treats software stacks and hardware as first-class experimental variables [3], [7], [8].

C. Packaging is measurable

The slim production image is 339.58 MiB uncompressed—89.2 $\times$ its 3.81 MiB model—and 81.43 MiB as a compressed image archive. The standard-base control is 1,543.65 MiB uncompressed and 393.44 MiB compressed, so the slim base reduces the two measures by 78.0% and 79.3%. Across 20 clean starts, median time from Compose invocation to the first successful health response is 1.52 s (range 1.39–1.83 s); ready-state RSS is 65.05–65.72 MiB.

After warm-up, 200 sequential localhost requests have mean 51.97 ms, p50 49.87 ms, p90 65.80 ms, and p99 73.96 ms. The additive mean decomposition is 38.66 ms for container invocation, 10.96 ms for client/loopback transport, and 2.35 ms for all other server work. The latter comprises framework/multipart parsing (0.67 ms), decode (0.30 ms), resize/crop (0.74 ms), normalize/quantize (0.26 ms), postprocessing (0.11 ms), serialization (0.04 ms), and 0.23 ms unattributed; upload read and queue wait round below 0.01 ms. Thus 31.23 ms of the 44.54 ms gap from 7.43 ms native to 51.97 ms HTTP mean occurs inside the container’s invocation boundary, while

TABLE III
CONTAINER AND END-TO-END SERVING MEASUREMENTS.

Metric	Measured value
Slim image, raw / compressed	339.58 / 81.43 MiB
Image / model, uncompressed	89.2×
Standard base, raw / compressed	1543.65 / 393.44 MiB
Cold start, median (range), $n = 20$	1.52 s (1.39–1.83)
HTTP latency, p50 / p99, $n = 200$	49.87 / 73.96 ms
RSS after 200 requests	77.16 MiB
Runtime limits	2 CPUs / 512 MiB

measured transport contributes 10.96 ms. Runtime/version, virtualization, and the two-CPU ceiling remain bundled within that invocation difference. RSS after requests is 77.16 MiB.

D. Threats to validity

The paired accuracy inference applies to this fixed 3,925-image subset; Imagenette does not replace full ImageNet evaluation. Latency was measured on one x86-64 workstation host, not ARM, a neural accelerator, or a battery-powered target. The deliberately interactive host condition and sampled pre-run CPU load differ from an idle laboratory benchmark, although 20 independent starts and paired intervals expose rather than hide that variation. The Windows-hosted Docker measurement includes virtualization and loopback transport and should not be generalized to native Linux. Energy, temperature, concurrent-request throughput, and multi-model generality remain unmeasured.

V. CONCLUSION

INT8 was reliably smaller, but it was not intrinsically faster or accurate. In this MobileNetV2 audit, per-channel quantization delivered useful accuracy and a 1.27× one-thread XNNPACK speedup, while per-tensor quantization destroyed accuracy and TensorFlow Lite’s built-in counterfactual made INT8 slower than FP32. The result replaces a blanket “quantization accelerates inference” narrative with a testable deployment rule: validate the model’s granularity and benchmark the exact backend, thread policy, and serving boundary that will run in production.

The repository turns that rule into a repeatable audit. It retains per-image paired predictions, process-level latency replicates, raw request timings, hashes, host controls, a hardened container, and the scripts that generated the tables and figure. Stronger external validity now requires replication across additional architectures, full ImageNet, native ARM systems, and energy-controlled edge hardware rather than additional claims from the present host.

DATA AND CODE AVAILABILITY

Source code, model artifacts, experiment scripts, and machine-readable results are available in the tagged release at <https://github.com/imjbassi/efficient-ai-edge-deployment/releases/tag/v1.1.0>. Repository-authored code and documentation use the MIT license; TensorFlow/Keras use Apache-2.0. Converted pretrained parameters are not relicensed by the artifact,

and Imagenette is downloaded from fastai, verified by its full archive SHA-256, and remains subject to ImageNet terms.

AUTHOR CONTRIBUTIONS

J. Bassi performed the conceptualization, methodology, software, investigation, data curation, visualization, and writing.

REFERENCES

- [1] B. Jacob, S. Kligys, B. Chen, M. Zhu, M. Tang, A. Howard, M. Andreetto, and D. Kalenichenko, “Quantization and training of neural networks for efficient integer-arithmetic-only inference,” in *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition*, 2018, pp. 2704–2713.
- [2] Google AI Edge, “Post-training integer quantization,” https://ai.google.dev/edge/litert/models/post_training_integer_quant, accessed: September 14, 2026.
- [3] V. J. Reddi, C. Cheng, D. Kanter, P. Mattson, G. Schmuelling, C.-J. Wu, B. Anderson, M. Breughe, M. Charlebois, W. Chou *et al.*, “MLPerf inference benchmark,” in *Proceedings of the 47th Annual International Symposium on Computer Architecture*, 2020, pp. 446–459.
- [4] Y. Li, M. Shen, J. Ma, Y. Ren, M. Zhao, Q. Zhang, R. Gong, F. Yu, and J. Yan, “MQBench: Towards reproducible and deployable model quantization benchmark,” in *Proceedings of the Neural Information Processing Systems Track on Datasets and Benchmarks*, 2021.
- [5] Y. Wang, K. Han, Y. Ma, X. Dai, Z. Li, S. Liu, P. Chen, Y. Li, Z. Wang, Z. Liu *et al.*, “SpaceEvo: Hardware-friendly search space design for efficient INT8 inference,” in *Proceedings of the IEEE/CVF International Conference on Computer Vision*, 2023, pp. 13 232–13 242.
- [6] M. Sandler, A. Howard, M. Zhu, A. Zhmoginov, and L.-C. Chen, “Mobilenetv2: Inverted residuals and linear bottlenecks,” in *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition*, 2018, pp. 4510–4520.
- [7] V. J. Reddi, D. Kanter, P. Mattson, J. Duke, T. Nguyen, R. Chukka, K. Shiring, K.-S. Tan, M. Charlebois, W. Chou *et al.*, “MLPerf mobile inference benchmark: Why mobile AI benchmarking is hard and what to do about it,” *arXiv preprint arXiv:2012.02328*, 2020.
- [8] C. Banbury, V. J. Reddi, P. Torelli, J. Holleman, N. Jeffries, C. Kiraly, P. Montino, D. Kanter, S. Ahmed, D. Pau *et al.*, “MLPerf tiny benchmark,” in *Proceedings of the Neural Information Processing Systems Track on Datasets and Benchmarks*, 2021.
- [9] R. Krishnamoorthi, “Quantizing deep convolutional networks for efficient inference: A whitepaper,” *arXiv preprint arXiv:1806.08342*, 2018.
- [10] J. Howard, “Imagenette: A smaller subset of 10 easily classified classes from ImageNet,” <https://github.com/fastai/imagenette>, accessed: September 14, 2026.
- [11] Q. McNemar, “Note on the sampling error of the difference between correlated proportions or percentages,” *Psychometrika*, vol. 12, no. 2, pp. 153–157, 1947.
- [12] B. Efron, “Bootstrap methods: Another look at the jackknife,” *The Annals of Statistics*, vol. 7, no. 1, pp. 1–26, 1979.